

Birla Institute of Technology and Science, Pilani

BITS F464: Machine Learning

Semester I, Academic Year 2026–27

Instructor: Prof. N L Bhanu Murthy

Assignment 1: Flat vs. Hierarchical Classification From Scratch

Due Date: 2355hrs, 26-10-2026

Demo & Viva for 20 mins per group on 27th, 28th and 29th of Oct (Mutual convenient timings)

Instructions & Rules

- **Strictly prohibited:** any machine-learning or deep-learning library, including scikit-learn, PyTorch, TensorFlow, XGBoost, LightGBM, SciPy and statsmodels, and library helpers such as train/test splits, metrics or confusion matrices. A prohibited import earns zero for that part.
- **Allowed:** Python 3, `numpy`, `pandas` and the standard library (`math`, `collections`, `itertools`, ...). `matplotlib` for optional plots only.
- **Dataset:** Anuran Calls (MFCCs), UCI Machine Learning Repository: https://archive.ics.uci.edu/ml/datasets/Anuran+Calls+(MFCCs). It has 7,195 records, 22 numeric features (`MFCCs_1` ... `MFCCs_22`) and the labels `Family` (4 classes), `Genus` (8), `Species` (10) and `RecordID`. **Only the 22 MFCC columns are inputs.** `Family`, `Genus`, `Species` and `RecordID` must never be used as features.
- **Two label levels.** *Flat* models predict `Species` directly (10 classes); *hierarchical* models predict `Family` first, then `Species`. Each species belongs to exactly one family. `Genus` is not used.
- **Train/test split (same for everyone).** Split the data 80/20, stratified on `Species` (and therefore approximately on `Family`), *using exactly the function below* with seed 42. It uses only `numpy`, so every student obtains the identical split. Put it in `common.py` and use the same split in all five parts. You should get roughly 5,756 training and 1,439 test rows (a few rows may differ because of per-species rounding). The test set is used *only* for final evaluation: no fitting, no tuning, no statistics computed from it.

```
def stratified_split(labels, test_frac=0.2, seed=42):
    rng = np.random.RandomState(seed)
    test = []
    for c in np.unique(labels):
        idx = rng.permutation(np.where(labels == c)[0])
        test.extend(idx[:int(round(test_frac * len(idx)))])
    test = np.sort(np.array(test))
    train = np.setdiff1d(np.arange(len(labels)), test)
    return train, test
```

- **Fixed hyperparameters, no tuning.** Use the settings in Table 1 so that all submissions are comparable.
- **Randomness and ties.** Apart from the split above (which has its own generator), create one fresh `np.random.RandomState(42)` per ensemble model and use it for all its randomness (bootstrap samples, feature subsets). Candidate thresholds are midpoints between consecutive distinct sorted feature values. If two splits tie on gain, choose the lower feature index, then the lower threshold. Ties in a leaf or in a vote go to the smallest class label (sorted order).
- **Leakage caveat.** Calls from the same recording appear in both train and test, so accuracies will be optimistic compared with unseen recordings. This is a property of the split, not a bug.

Table 1: Fixed hyperparameters (all trees use `min_samples_split=2`; $d = 22$ features, $N \approx 5756$ training rows).

Method	Settings
Single tree	`max_depth=None` (unconstrained).
Bagging	$T = 25$ trees; bootstrap sample of size N; all 22 features at every split; unconstrained; majority vote.
AdaBoost	$T = 50$ rounds; weak learner = tree with `max_depth=3`; multi-class SAMME (Part 2).
Random Forest	$T = 50$ trees; bootstrap of size N; $m = \lfloor\sqrt{22}\rfloor = 4$ features drawn *afresh at every split*; unconstrained; majority vote.
Hierarchical tree	Router and every expert: one unconstrained tree, same criterion.
Gaussian NB	Variance smoothing: add $10^{-9} \cdot \max_j \mathrm{Var}_{\mathrm{train}}(x_j)$ to every variance.

Part 1: Single Decision Tree (Gini and Information Gain)

This is the building block for Parts 2–4. Build it carefully, because everything else reuses it.

Requirements

Implement a class `DecisionTree(criterion, max_depth, min_samples_split, max_features=None, rng=None)` with methods `fit(X, y, sample_weight=None)` and `predict(X)`. The last three arguments are used in Parts 2 and 3; add them now so you do not have to rewrite the tree later. `criterion` is `"gini"` or `"entropy"`, chosen at instantiation.

With sample weights w_n (all equal to $1/N$ by default), the class probability at a node $\mathcal{N}$ is $p_i = \sum_{n\in\mathcal{N}} w_n \mathbf{1}[y_n = i] \,/\, \sum_{n\in\mathcal{N}} w_n$. Then

$$G(\mathcal{N}) = 1 - \sum_{i=1}^{C} p_i^2, \qquad H(\mathcal{N}) = -\sum_{i=1}^{C} p_i \log_2 p_i \quad (0 \log_2 0 := 0),$$

$$\text{Gain} = \text{Imp}(\text{parent}) - \sum_{j\in\{L,R\}} \frac{W_j}{W} \text{Imp}(\text{child}_j),$$

where Imp is G (Gini decrease) or H (information gain) and W_j is the total weight in child j.

- **Splits:** binary, of the form $x_j \le t$. At each node evaluate every feature (or the sampled subset, Part 3) and every candidate threshold, and take the split with the largest gain.
- **Stopping:** the node is pure, `max_depth` is reached, the node has fewer than `min_samples_split` samples, or no split has positive gain. The leaf predicts the (weighted) majority class.
- **Efficiency:** sort each feature once per node and use cumulative class counts, so all thresholds are scored at once. A Python loop over thresholds will be far too slow. A vectorised implementation fits an unconstrained tree on the full training set in a few seconds.

Sanity checks (pass these before moving on)
(a) $G = 0.5$ and $H = 1$ bit for class counts $(3, 3)$. (b) $X = [[1], [2], [3], [4]]$, $y = [0, 0, 1, 1]$ must give one split at $t = 2.5$ with information gain 1 and training accuracy 1.0. (c) Three identical samples with labels $[0, 0, 1]$ and weights $[0.1, 0.1, 0.8]$ must give a leaf predicting class 1. (d) A pure node, or a family with a single class, must not crash (the tree is just one leaf).

Task

1. Train a **flat** tree on the 22 MFCC features to predict `Species`, once with Gini and once with entropy (Table 1 settings). Report training and test accuracy for both.
2. From the predicted species, derive the predicted family using the species→family mapping read from the training data, and report the *derived family accuracy* on the test set. (Part 4 compares this against the router.)
3. Print, for each criterion, train and test accuracy for `max_depth` $\in \{1, 2, 3, 5, 8, 12, \text{None}\}$. This is for your own understanding of bias and variance and is not part of the matrix. You should see training accuracy reach 100% with an unconstrained tree.

Fill the **Single Decision Tree** rows of the results matrix (page 8).

Part 2: Bagging and Boosting on Flat Trees

Both ensembles must use your `DecisionTree` from Part 1 as the base estimator, predict `Species` (flat, $K = 10$ classes), and be run with both Gini and entropy.

2.1 Bagging

1. For $t = 1, \ldots, T$ ($T = 25$): draw a bootstrap sample of N rows with replacement (from your `RandomState(42)`) and fit an unconstrained tree on it, using *all* 22 features at every split.
2. Predict by majority vote over the T trees (ties $\rightarrow$ smallest class label).

2.2 AdaBoost (multi-class SAMME)

The binary update $\exp(-\alpha_t y_i h_t(x_i))$ assumes labels in $\{-1, +1\}$ and does not apply to $K = 10$ classes. Implement the multi-class generalisation SAMME instead. Implement the weight updates yourself.

1. Initialise $w_i = 1/N$.
2. For $t = 1, \ldots, T$ ($T = 50$): fit a tree with `max_depth=3` on the training data using the sample weights w (your tree must honour `sample_weight`; see Part 1).
3. Weighted error: $\epsilon_t = \sum_i w_i \, \mathbf{1}[h_t(x_i) \neq y_i]$ (with $\sum_i w_i = 1$). Clip ϵ_t to $[10^{-10}, 1 - 10^{-10}]$. If $\epsilon_t \geq 1 - 1/K$, the learner is no better than random guessing: discard it and stop boosting.
4. Amount of say: $\alpha_t = \ln \dfrac{1 - \epsilon_t}{\epsilon_t} + \ln(K - 1)$.
5. Update: $w_i \leftarrow w_i \exp\big(\alpha_t \, \mathbf{1}[h_t(x_i) \neq y_i]\big)$, then normalise so that $\sum_i w_i = 1$.
6. Final prediction: $H(x) = \arg\max_k \sum_t \alpha_t \, \mathbf{1}[h_t(x) = k]$.

Check: for $K = 2$ this is the usual AdaBoost with α doubled, which gives the same classifier.

Implementation notes

- Bootstrap by drawing indices, `rng.randint(0, N, size=N)`, and indexing the arrays. Do not copy data in a Python loop.
- For the weighted tree, replace class *counts* by class *weight sums* everywhere (impurity, gain, leaf majority).
- Keep each trained tree's α_t and prediction function; the ensemble predict function must work on any X (train or test).

Task

1. Train Bagging and AdaBoost with each criterion. Report train accuracy, test accuracy, and the derived family accuracy on the test set.
2. Compare test accuracy with the single tree of Part 1.
3. Additionally, run AdaBoost with decision stumps (`max_depth=1`, $T = 50$) and print its test accuracy next to the `max_depth=3` result. This is not part of the matrix. Consider why stumps struggle on a 10-class problem.

Fill the **Bagging** and **AdaBoost** rows of the results matrix.

Part 3: Random Forest on Flat Trees

A Random Forest is bagging plus *feature subsetting*. It predicts `Species` directly and is run with both Gini and entropy.

Requirements

Implement `RandomForest(n_estimators, criterion, max_features, ...)` using your `DecisionTree`.

- **Bootstrapping:** each tree is trained on its own bootstrap sample of N rows drawn with replacement.
- **Feature subsetting at every split:** at *each node* of *each tree*, draw $m = \lfloor \sqrt{d} \rfloor = 4$ candidate features without replacement from the 22 (use `rng.choice(d, m, replace=False)`) and search for the best split among only those. Drawing one subset per tree is a different (and wrong) algorithm. Your `DecisionTree` needs the `max_features` and `rng` arguments from Part 1.
- If none of the m sampled features yields a split with positive gain, the node becomes a leaf.
- **Prediction:** majority vote over $T = 50$ unconstrained trees (ties $\rightarrow$ smallest class label).

Why feature subsetting?
Bagged trees are trained on overlapping data and tend to pick the same strong features near the root, so their errors are correlated and averaging helps less. Restricting each split to a random subset of features forces the trees to differ, which reduces the correlation between them. This is the main idea of this part.

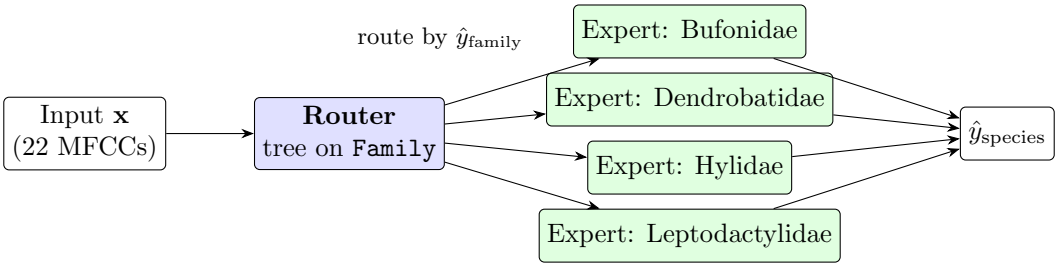
Task

1. Train the Random Forest with each criterion. Report train accuracy, test accuracy, and derived family accuracy on the test set.
2. Compare test accuracy with the single tree (Part 1) and with Bagging (Part 2).
3. *Extra (not graded):* measure the average pairwise agreement of the individual trees' test predictions for Bagging and for the Random Forest (fraction of test samples on which two trees predict the same class, averaged over all pairs of trees). Is the forest's agreement lower, as the explanation above suggests?

Fill the **Random Forest** rows of the results matrix.

Background: Hierarchical Decision Trees

Idea. Class labels often form a tree. Here, species are grouped into families. A *flat* classifier ignores this and separates all 10 species with one model. A *hierarchical* classifier follows the label tree top-down (the "local classifier per parent node" approach surveyed in [2]): a **router** at the root predicts the family, and a family-specific **expert** then predicts the species within that family.



Training. The router is trained on *all* training samples with the `Family` label. Each expert is trained only on the training samples whose *true* family is its own, with the `Species` label.
Inference. Route **x** with the router, send it to the expert of the predicted family, and return that expert's species.

When can hierarchy beat a flat model?

- *Specialisation:* each expert solves a smaller problem (fewer classes) using the features that matter inside its family; the router only separates coarse groups.
- *Theory:* Babbar et al. [1] derive generalisation bounds for flat and hierarchical classifiers in large-scale taxonomies. In their analysis the capacity term favours the hierarchy, but top-down classification pays for error propagation; which effect dominates depends on the taxonomy and on the quality of the node classifiers.

When can it lose? A sample sent to the wrong family can never receive the right species, because every species belongs to exactly one family. Experts also see fewer training samples. So the hierarchy helps only if the gain from easier sub-problems outweighs the loss from router mistakes. *This is an empirical question: the results matrix you fill in answers it for this dataset.*

Metrics. On the test set of N samples, let $y_n^F, \hat{y}_n^F$ be the true and router-predicted family and $y_n^S, \hat{y}_n^S$ the true and final predicted species.

- **Router accuracy:** $A_{\text{router}} = \dfrac{1}{N}\sum_n \mathbf{1}[\hat{y}_n^F = y_n^F]$.
- **Overall accuracy:** $A_{\text{overall}} = \dfrac{1}{N}\sum_n \mathbf{1}[\hat{y}_n^S = y_n^S]$ (the final species accuracy).
- **Oracle accuracy** A_{oracle}: the overall accuracy obtained when each sample is sent to the expert of its *true* family (a perfect router).

Errors fall into two failure modes: **(A)** router correct but expert wrong, with fraction p_A; **(B)** router wrong (the species is then always wrong), with fraction $p_B = 1 - A_{\text{router}}$. Hence

$$A_{\text{overall}} = 1 - p_A - p_B = A_{\text{router}} - p_A \;\leq\; A_{\text{router}}.$$

A_{oracle} measures the experts alone; the gap $A_{\text{oracle}} - A_{\text{overall}}$ is the price of router errors.

References.

[1] R. Babbar, I. Partalas, E. Gaussier, M.-R. Amini. *On Flat versus Hierarchical Classification in Large-Scale Taxonomies.* Advances in Neural Information Processing Systems 26 (NIPS 2013), pp. 1824–1832. https://proceedings.neurips.cc/paper/2013/hash/cbb6a3b884f4f88b3a8e3d44c636cbd8-Abstract.html

[2] C. N. Silla Jr., A. A. Freitas. *A Survey of Hierarchical Classification Across Different Application Domains.* Data Mining and Knowledge Discovery 22:31–72, 2011.

Part 4: Hierarchical Decision Tree (Gini and Information Gain)

Combine single trees into the router-plus-experts pipeline described on the previous page. Run the whole pipeline once with Gini and once with entropy (router and experts use the same criterion).

Requirements

1. **Router.** Train a `DecisionTree` on all training samples to predict `Family`.
2. **Experts.** Partition the *training* data by its *true* `Family` (4 subsets: Bufonidae, Dendrobatidae, Hylidae, Leptodactylidae). Train one `DecisionTree` per subset to predict `Species`. Store them in a dictionary `{family_name: trained_expert}`. Inspect the data: some families contain only a single species, and their expert must still work (a constant predictor).
3. **Classifier.** Implement `HierarchicalClassifier(router, experts)` with `predict_family(X)` (router output) and `predict(X)`. For `predict`: route all samples, then group them by predicted family and call each expert *once* on its group (no per-sample Python loops over the models), and return the species for every row in the original order.

Evaluation

For each criterion, on the test set (and training set where indicated), compute and print:

- **Router accuracy** A_{router} (train and test) and **overall accuracy** A_{overall} (train and test);
- the oracle accuracy A_{oracle}, and the failure fractions p_A (router right, expert wrong) and p_B (router wrong);
- a 4×4 confusion matrix for the router and a 10×10 confusion matrix for the final species prediction (implement it with `numpy`), and the per-family test accuracy;
- a numerical check that $A_{\text{overall}} = A_{\text{router}} - p_A$.

Comparison with the flat model

Compare, for the same criterion, the hierarchical model with the single flat tree of Part 1: (i) overall species accuracy against the flat species accuracy, and (ii) router accuracy against the flat tree's derived family accuracy. Is the router or the experts the bottleneck? Does the hierarchy help on this dataset?

Common mistakes

- Training experts on the *predicted* family. Experts are trained on the true family; the router's mistakes only matter at test time.
- Passing the same unchanged feature rows through all experts and then keeping one answer: route first, then predict.
- Returning predictions in grouped order instead of the original sample order.

Fill the **Hierarchical Decision Tree** rows of the results matrix. The "Family acc." cell for these rows is the *router accuracy.*

Part 5: Flat Gaussian Naive Bayes

Naive Bayes is a probabilistic model that is not tree-based. Implement it for the flat problem (predict `Species`). It has no splitting criterion.

Model

Assume the 22 features are conditionally independent given the class and, within a class, Gaussian. From the training data, estimate for every class c and feature j (with n_c samples in class c):

$$\pi_c = \frac{n_c}{N}, \qquad \mu_{cj} = \frac{1}{n_c} \sum_{n:\, y_n = c} x_{nj}, \qquad \sigma_{cj}^2 = \frac{1}{n_c} \sum_{n:\, y_n = c} (x_{nj} - \mu_{cj})^2 + \varepsilon,$$

where $\varepsilon = 10^{-9} \cdot \max_j \mathrm{Var}_{\text{train}}(x_j)$ (variance smoothing, Table 1). Predict with

$$\hat{y} = \arg\max_c \Big[\log \pi_c - \frac{1}{2} \sum_{j=1}^{22} \Big(\log\left(2\pi\sigma_{cj}^2\right) + \frac{(x_j - \mu_{cj})^2}{\sigma_{cj}^2} \Big) \Big].$$

Requirements

- Implement `GaussianNB(var_smoothing=1e-9)` with `fit(X, y)` and `predict(X)`.
- Work in **log space**. Multiplying 22 densities directly can underflow to zero.
- Vectorise: compute the log-likelihood of all test samples for all classes with `numpy` broadcasting (shape $n \times K \times d$ is small enough), not with Python loops over samples.

Sanity check
One feature, class 0 with values $\{0, 2\}$ and class 1 with values $\{10, 12\}$ (equal priors): the means are 1 and 11 and the variances are 1 (plus ε). Then $x = 3$ must be predicted as class 0 and $x = 9$ as class 1.

Task

1. Train on the training set and report train accuracy and test accuracy for `Species`, and the derived family accuracy on the test set (same species→family mapping as in Part 1).
2. Compare with the tree-based models. Naive Bayes assumes conditional independence of features; compute the correlation matrix of the 22 training features and think about how the correlation between MFCCs may affect the model.

Fill the **Gaussian Naive Bayes** row of the results matrix.

Deliverables & Results Matrix

Submit **one zip file** named `<StudentID>_A1.zip` containing:

- the source code for **every part** (`part1_decision_tree.py`, `part2_bagging_boosting.py`, `part3_random_forest.py`, `part4_hierarchical.py`, `part5_naive_bayes.py`; shared code may go in `common.py`). Each script must run with one command from a folder containing `Frogs_MFCCs.csv` (no absolute paths) and print the numbers that go into the matrix;
- `README.txt`: Python/numpy/pandas versions and the commands to run;
- `results_matrix.pdf`: **the filled-in accuracy matrix below**, typed (not scanned), accuracies in % with two decimals.

Table 2: Results matrix to fill in (flat models predict `Species`; † see note).

Method	**Criterion**	**Overall accuracy (Species)**		**Family acc. †**
		Train	**Test**	**Test**
1. Single Decision Tree	Gini			
	Entropy			
2a. Bagging	Gini			
	Entropy			
2b. AdaBoost	Gini			
	Entropy			
3. Random Forest	Gini			
	Entropy			
4. Hierarchical Decision Tree	Gini			**Router:**
	Entropy			**Router:**
5. Gaussian Naive Bayes	—			

† For flat models, the *derived* family accuracy (map each predicted species to its family). For the Hierarchical rows, write the **router accuracy** in the shaded cells next to "Router:" (the "Overall accuracy" columns hold the final species accuracy).

Table 3: Supplementary: hierarchical failure analysis (test set, %).

Criterion	**Router acc.**	**Oracle acc.**	**(A) Router right, expert wrong**	**(B) Router wrong**
Gini				
Entropy				

Help. If you get stuck, email either of the teaching assistants:

- f20231091@hyderabad.bits-pilani.ac.in
- f20230390@hyderabad.bits-pilani.ac.in

Submissions are checked for similarity. The code you submit must be your own work.